# Observability: логирование, мониторинг и планирование задач

## Table of contents

Observability: логирование, мониторинг и планирование задач . . . 1
Три столпа Observability . . . 1
Структурированное логирование . . . 2
Prometheus: сбор метрик . . . 3
Grafana: визуализация и дашборды . . . 5
Distributed Tracing (Jaeger, OpenTelemetry) . . . 6
Планировщики задач . . . 8

## Observability: логирование, мониторинг и планирование задач

После того как система защищена и запущена в продакшене, возникает вопрос: **как понять, что происходит внутри?**

Когда приложение работает на вашем компьютере, вы видите вывод в консоль, можете поставить breakpoint и посмотреть переменные. Но в продакшене приложение работает на сервере, обрабатывает тысячи запросов в секунду, и вы не можете просто «остановить и посмотреть». Более того, если пользователь жалуется «у меня не работает», вам нужно понять, **что именно** пошло не так, **когда** это произошло и **почему**.

**Observability** (наблюдаемость) — это способность понять внутреннее состояние системы по её внешним выходным данным. Три столпа observability — это **логи** (что произошло), **метрики** (сколько и как быстро) и **трейсы** (путь запроса через систему).

Эта лекция также охватывает **планирование задач** — механизмы запуска фоновых задач и построения конвейеров обработки данных. В продакшене многие операции выполняются асинхронно: отправка email, обработка загруженных файлов, построение отчётов. Вместо того чтобы заставлять пользователя ждать, мы ставим задачу в очередь и обрабатываем её фоном. Celery, Airflow и Dagster решают эту проблему на разных уровнях абстракции.

### Три столпа Observability

**Логи (Logs):** дискретные события (“user123 logged in at 14:30”)

**Метрики (Metrics):** агрегированные числовые данные (CPU usage: 65%, requests/sec: 1500)

**Трейсы (Traces):** путь запроса через микросервисы (request → API Gateway → Auth → Database)

## Структурированное логирование

**Проблема традиционных логов:**

```
2025-01-02 14:30:15 ERROR Failed to process order: user not found
2025-01-02 14:30:16 INFO User alice@company.com logged in from 192.168.1.10
```

- Трудно парсить (нужны regex)
- Нет структуры для фильтрации
- Сложно аггрегировать

**Структурированные логи (JSON):**

```
{
  "timestamp": "2025-01-02T14:30:15Z",
  "level": "ERROR",
  "message": "Failed to process order",
  "error": "user not found",
  "user_id": "user-123",
  "order_id": "order-456",
  "service": "order-service",
  "trace_id": "abc-def-ghi"
}

{
  "timestamp": "2025-01-02T14:30:16Z",
  "level": "INFO",
  "message": "User logged in",
  "user_email": "alice@company.com",
  "ip": "192.168.1.10",
  "service": "auth-service",
  "trace_id": "xyz-123-456"
}
```

**Преимущества:**

- Легко фильтровать: `service="order-service" AND level="ERROR"`
- Аггрегация: сколько ошибок по сервисам?
- Корреляция: trace_id связывает логи одного запроса

**Популярные библиотеки:**

- **Go:** zap, logrus
- **Python:** structlog, python-json-logger
- **Node.js:** winston, pino

**Централизованный сбор логов:**

- **ELK Stack:** Elasticsearch (хранение) + Logstash (обработка) + Kibana (визуализация)
- **Loki** (от Grafana Labs): как Prometheus, но для логов
- **Splunk:** enterprise-решение

## Prometheus: сбор метрик

**Что это:** база данных временных рядов для метрик. Prometheus сам забирает метрики из приложений по pull-модели.

**Типы метрик:**

1. **Counter** — монотонно растущий счётчик (сбрасывается при рестарте)

```
Пример: http_requests_total (общее число запросов)
Использование: вычисление rate (запросов в секунду)
```

2. **Gauge** — текущее значение (может расти и падать)

```
Пример: memory_usage_bytes, active_connections
```

3. **Histogram** — распределение значений по бакетам

```
Пример: http_request_duration_seconds (сколько запросов заняло <0.1s, <0.5s,
<1s, <5s)
Использование: вычисление percentiles (p50, p95, p99)
```

4. **Summary** — похож на Histogram, но percentiles вычисляются на клиенте

```
Пример: request_duration_summary{quantile="0.95"}
```

**Пример инструментирования (Go):**

```go
import "github.com/prometheus/client_golang/prometheus"

var (
    httpRequestsTotal = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "endpoint", "status"},
    )

    httpRequestDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
```

```go
            Name:    "http_request_duration_seconds",
            Help:    "HTTP request duration in seconds",
            Buckets: []float64{0.01, 0.05, 0.1, 0.5, 1.0, 5.0},
        },
        []string{"method", "endpoint"},
    )
)

func init() {
    prometheus.MustRegister(httpRequestsTotal)
    prometheus.MustRegister(httpRequestDuration)
}

func handleRequest(w http.ResponseWriter, r *http.Request) {
    start := time.Now()

    // ... обработка запроса ...

    duration := time.Since(start).Seconds()
    status := 200

                    httpRequestsTotal.WithLabelValues(r.Method,     r.URL.Path,
fmt.Sprint(status)).Inc()
    httpRequestDuration.WithLabelValues(r.Method, r.URL.Path).Observe(duration)
}
```

**Prometheus scraping:**

Приложение expose /metrics endpoint:

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",endpoint="/api/users",status="200"} 15234
http_requests_total{method="POST",endpoint="/api/users",status="201"} 482
http_requests_total{method="GET",endpoint="/api/users",status="404"} 12

# HELP http_request_duration_seconds HTTP request duration in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{method="GET",endpoint="/api/
users",le="0.01"} 8521
http_request_duration_seconds_bucket{method="GET",endpoint="/api/
users",le="0.05"} 14102
http_request_duration_seconds_bucket{method="GET",endpoint="/api/
users",le="0.1"} 15000
```

```
http_request_duration_seconds_bucket{method="GET",endpoint="/api/
users",le="0.5"} 15200
http_request_duration_seconds_bucket{method="GET",endpoint="/api/
users",le="+Inf"} 15234
http_request_duration_seconds_sum{method="GET",endpoint="/api/users"} 652.3
http_request_duration_seconds_count{method="GET",endpoint="/api/users"} 15234
```

Prometheus config (prometheus.yml):

```yaml
scrape_configs:
  - job_name: 'my-app'
    scrape_interval: 15s
    static_configs:
      - targets: ['localhost:8080']
```

**PromQL (Prometheus Query Language) — примеры:**

```
# Текущее число активных соединений
active_connections

# Запросов в секунду за последние 5 минут
rate(http_requests_total[5m])

# Запросов в секунду по эндпоинтам
sum(rate(http_requests_total[5m])) by (endpoint)

# P95 latency (95% запросов быстрее этого значения)
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Процент ошибок (5xx)
sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
sum(rate(http_requests_total[5m]))

# Alerting: если error rate > 5%
sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
sum(rate(http_requests_total[5m]))
  > 0.05
```

## Grafana: визуализация и дашборды

**Что это:** платформа для визуализации метрик из Prometheus, Loki, Elasticsearch и др.

**Пример дашборда для веб-сервиса:**

Панели:

1. **QPS (Queries Per Second)** — graph

```
sum(rate(http_requests_total[1m]))
```

2. **Latency (P50, P95, P99)** — graph

```
histogram_quantile(0.50, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

3. **Error Rate** — single stat + graph

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m])) * 100
```

4. **Top endpoints by QPS** — table

```
topk(10, sum(rate(http_requests_total[5m])) by (endpoint))
```

5. **Memory usage** — graph

```
process_resident_memory_bytes
```

**Alerting в Grafana:**

Настройка алерта “High Error Rate”:

```
WHEN avg() OF query(A, 5m, now) IS ABOVE 0.05
SEND TO: [Slack, PagerDuty, Email]
```

## Distributed Tracing (Jaeger, OpenTelemetry)

**Проблема:** запрос проходит через 5 микросервисов, где узкое место?

**Решение:** трейсинг показывает путь и время каждого шага.

**Пример запроса:**

```
User → API Gateway → Auth Service → User Service → Database
                   ↘ Cache
```

**Trace:**

```
Trace ID: abc-123-xyz
Total duration: 245ms

Span 1: API Gateway (245ms total)
  ├─ Span 2: Auth Service (50ms)
  │    └─ Span 3: Cache GET (5ms)
  └─ Span 4: User Service (180ms)
       └─ Span 5: Database SELECT (170ms) ← bottleneck!
```

**Инструментирование (OpenTelemetry, Go):**

```go
import "go.opentelemetry.io/otel"

func handleRequest(ctx context.Context, userID string) error {
    ctx, span := otel.Tracer("api-gateway").Start(ctx, "handleRequest")
    defer span.End()

    // Вызов auth service
    if err := checkAuth(ctx, userID); err != nil {
        span.RecordError(err)
        return err
    }

    // Вызов user service
    user, err := getUserData(ctx, userID)
    if err != nil {
        span.RecordError(err)
        return err
    }

    span.SetAttributes(attribute.String("user.email", user.Email))
    return nil
}

func getUserData(ctx context.Context, userID string) (*User, error) {
    ctx, span := otel.Tracer("user-service").Start(ctx, "getUserData")
    defer span.End()

    // Запрос к БД
    user, err := db.Query(ctx, "SELECT * FROM users WHERE id = ?", userID)
    // ...
    return user, err
}
```

**Trace распространяется через HTTP headers:**

```
traceparent: 00-abc123xyz-def456-01
             |  |         |       └─ flags (1 байт)
             |  |         └─ span ID (8 байт = 16 hex символов)
             |  └─ trace ID (16 байт = 32 hex символа)
             └─ version (1 байт = 2 hex символа)
```

**Популярные решения:**

- **Jaeger** — open-source от Uber
- **Zipkin** — open-source от Twitter
- **OpenTelemetry** — стандарт для инструментирования (заменяет OpenTracing + OpenCensus)
- **Datadog APM, New Relic** — SaaS-решения

## Планировщики задач

### 1. Celery (Python task queue)

**Что это:** асинхронная очередь задач для Python.

**Архитектура:**

```
Producer → Broker (Redis/RabbitMQ) → Worker → Result Backend
```

**Пример:**

```python
from celery import Celery

app = Celery('tasks', broker='redis://localhost:6379/0')

@app.task
def send_email(user_email, subject, body):
    # Отправка email (медленная операция)
    smtp.send(user_email, subject, body)
    return f"Email sent to {user_email}"

# Использование
# Синхронно (блокирует):
# send_email("alice@example.com", "Hello", "Welcome!")

# Асинхронно (возвращает сразу):
result = send_email.delay("alice@example.com", "Hello", "Welcome!")
# result.id → task ID для отслеживания
```

**Сценарии использования:**

- Отправка email/SMS после регистрации
- Генерация отчётов (PDF, Excel)
- Обработка загруженных изображений (resize, watermark)
- Очистка старых данных (cron-задачи)

**Periodic tasks (cron):**

```
from celery.schedules import crontab

app.conf.beat_schedule = {
    'cleanup-old-sessions': {
        'task': 'tasks.cleanup_sessions',
        'schedule': crontab(hour=2, minute=0),  # каждый день в 02:00
    },
}
```

**2. Airflow (оркестрация рабочих процессов)**

**Что это:** платформа для создания, планирования и мониторинга **DAG** (Directed Acyclic Graphs) — конвейеров обработки данных.

**Пример DAG:**

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

dag = DAG(
    'user_analytics_pipeline',
    start_date=datetime(2025, 1, 1),
    schedule_interval='@daily',  # запускать каждый день
)

def extract_data():
    # Скачать данные из S3
    pass

def transform_data():
    # Обработать данные (Spark, Pandas)
    pass

def load_to_warehouse():
    # Загрузить в ClickHouse
    pass
```

```
extract_task = PythonOperator(task_id='extract', python_callable=extract_data,
dag=dag)
transform_task = PythonOperator(task_id='transform',
python_callable=transform_data, dag=dag)
load_task = PythonOperator(task_id='load', python_callable=load_to_warehouse,
dag=dag)

extract_task >> transform_task >> load_task  # зависимости
```

**Визуализация:**

```
[Extract] → [Transform] → [Load]
     ↓           ↓            ↓
  Success    Success      Success
```

**Сценарии использования:**

- ETL/ELT-конвейеры (данные из источников → обработка → хранилище)
- ML-конвейеры обучения (подготовка данных → обучение → развёртывание модели)
- Бэкапы и синхронизация данных

**3. Dagster (оркестратор данных)**

**Что это:** современная альтернатива Airflow с фокусом на **ресурсы, описанные в коде**.

**Отличие от Airflow:**

- **Airflow:** описываем **задачи**
- **Dagster:** описываем **ресурсы** (assets) и их зависимости

**Пример:**

```
from dagster import asset

@asset
def raw_user_events():
    # Скачать события из Kafka
    return pd.read_csv("s3://bucket/events.csv")

@asset
def cleaned_events(raw_user_events):
    # Зависит от raw_user_events
    return raw_user_events.dropna()

@asset
def user_metrics(cleaned_events):
```

```
    # Вычислить метрики
    return cleaned_events.groupby('user_id').agg({'revenue': 'sum'})
```

**Преимущества:**

- Типизация данных между шагами
- Автоматическое кеширование (не пересчитываем, если данные не изменились)
- Встроенный data lineage (откуда данные?)

**Сценарии использования:**

- Современные конвейеры обработки данных
- ML-конвейеры с версионированием данных
- Обработка в реальном времени + пакетная обработка

**Сравнение планировщиков:**

| Характеристика | Celery | Airflow | Dagster |
|---|---|---|---|
| Тип | Очередь задач | Оркестратор рабочих процессов | Оркестратор данных |
| Фокус | Асинхронные задачи | ETL и пакетные конвейеры | Ресурсы данных и lineage |
| Сложность | Низкая | Средняя | Средняя |
| UI | Flower (опционально) | Встроенный | Встроенный (Dagit) |
| Retry/backoff | Да | Да | Да |
| Когда использовать | Фоновые задачи в веб-приложениях | Конвейеры обработки данных, ETL | Современные конвейеры обработки данных, ML |